

Linking Files in GitHub README

https://gemini.google.com/app/761bef1b7ed4abbb

User prompt: In a README file in GitHub, how can I include a link to another document in the same repository so that someone can just click on some text and then gets redirected to that other file?

Response: Use standard Markdown link syntax with a **relative path** to the target file:

[Text people will click on](path/to/your/file.md)

Using relative paths instead of full https://github.com/... URLs ensures your links keep working if someone forks the repo, switches branches, or renames the repository.

Common Examples

File Location	Markdown Syntax
Same folder as the README	[Read the setup guide](SETUP.md) or [Read the setup guide](./SETUP.md)
Inside a subfolder (e.g., docs/)	[API Documentation](docs/api.md)
One folder up (if your README is in a subfolder)	[License](../LICENSE)
Root of the repository (from anywhere)	[Contributing Guidelines](/CONTRIBUTING.md)
A code file or notebook	[See the preprocessing script](src/preprocess.py)

Linking to a Specific Section or Line

You can jump directly to a heading inside another Markdown file or a specific line of code:

- **To a heading in another .md file:** Add # followed by the heading name in lowercase, with spaces replaced by hyphens:

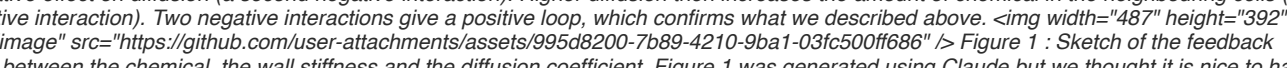
[See the installation steps](SETUP.md#installation-steps)

- **To a specific line in a code file:** Add #L followed by the line number (or #L10-L20 for a range):

[Check the model configuration](src/config.py#L42)

Tip for spaces in filenames: If the target filename has spaces, either replace the spaces with %20 or wrap the path in angle brackets so Markdown doesn't break the link: [Project Report](<docs/Project Report.pdf>) or [Project Report](docs/Project%20Report.pdf)

User prompt: # Assignment 5 KEN3170 Plant Tissue Simulations: Assignment ## Question 1 We opened the Infection model, which loads pathogen_infection.xml, and ran it for 2 hours of simulated time. We took a screenshot at the start and then every 30 minutes. At the start, the pathogen (the red cell) sits just outside the left edge of the tissue and none of the plant cells are infected yet. The healthy cells are light blue, and the cell files on the right side are green. As the simulation runs, the cells closest to the pathogen turn purple because the chemical produced by the pathogen diffuses into them. The infection first spreads along the outermost cell file on the left, the one in contact with the pathogen, from top to bottom, and then moves inward more slowly, one cell file at a time. After 2 hours, most of the first two cell files (roughly a quarter of the tissue) are purple and some cells in the third file are starting to change colour. The green cell files on the right are not affected at all. The tissue also deforms. The pathogen keeps growing, so it takes up more space and pushes against the plant cells around it. Because the infected cells have weaker walls, they are deformed more easily and get squeezed, while cells further away mostly keep their shape. The cells also get rounder and the walls look thicker between 0 and 30 minutes, but this happens in the whole tissue, also far away from the pathogen, so it is only the tissue mechanics settling at the start. The deformation caused by the infection is local, around the pathogen. ## Question 2 In CellHouseKeeping, each plant cell first looks at its chemical concentration. This value is divided by 0.5 and capped at 1.2, which gives a sort of infection level. If this infection level is above 0.1 and the cell is not the pathogen, the stiffness of all its wall elements is set to 3 minus the infection level. If the level is 0.1 or lower, the cell just keeps the normal stiffness of 3. So the more chemical a cell has, the softer its walls get, and this relation is linear. Because of the cap at 1.2, the stiffness can never go lower than 1.8. Infected cells also lose their veto, which is relevant for question 5. This matches what was explained in the lecture, where fungi and bacteria produce chemicals that weaken the plant cell wall to make it easier to infect the tissue. The pathogen behaves differently in a few ways. Its own walls are never weakened, because the weakening rule skips cell type 2, so it always keeps a stiffness of 3. It is also the only cell that grows: every step its target area increases by 2, and when its area becomes larger than rel_cell_div_threshold times the base area, it divides. The base area is a reference value that is the same for all cells (1000 in pathogen_infection.xml), not the pathogen's own starting size, which is about 559. Finally, in CellDynamics the pathogen produces the chemical at a constant rate, while the plant cells only slowly degrade it. So the pathogen is the source of the chemical and the plant cells are the ones affected by it. ## Question 3 In CelloCellTransport the chemical moves between neighbouring cells through passive diffusion, in a similar way to auxin in the exercise during the computer practical. The flux depends on the length of the wall, the diffusion coefficient and the difference in concentration between the two cells, with a small correction for the cell areas. The difference with the auxin model is that the diffusion coefficient is not a fixed parameter. First, the function getLengthAndStiffness calculates the average stiffness of the wall between the two cells, weighted by the length of the wall elements and taking both sides of the wall into account. The diffusion coefficient is then 0.00001 divided by this average stiffness. This means that a softer wall lets the chemical pass through faster. If the stiffness is almost zero, the code just uses 0.00001 so it does not divide by a number close to zero. This creates a feedback loop. When a cell receives the chemical, its walls become softer (as was also discussed in question 2). Softer walls give a higher diffusion coefficient, so the chemical moves faster through

those walls into the next cells. These cells then also get softer walls and pass the chemical on even faster. Each step strengthens the next one, so this is a positive feedback loop, and the infection basically speeds up its own spread. It does not grow without limit, because the stiffness cannot go below $3 - 1.2 = 1.8$. The diffusion coefficient therefore ranges from $0.00001/3 \approx 3.3 \cdot 10^{-6}$ (healthy wall) to $0.00001/1.8 \approx 5.6 \cdot 10^{-6}$ (fully weakened wall), so it can increase by at most a factor $3/1.8 \approx 1.67$. On top of that, plant cells degrade the chemical at a rate of $0.001 \times$ their chemical level (which can be seen in the method called *CellDynamics*). We can also check the sign of the loop with what we saw about feedback loops in the network biology lecture, where a loop is positive if it contains an even number of negative interactions. Here, more chemical lowers the stiffness (a negative interaction), and since the diffusion coefficient is 0.00001 divided by the stiffness, stiffness also has a negative effect on diffusion (a second negative interaction). Higher diffusion then increases the amount of chemical in the neighbouring cells (a positive interaction). Two negative interactions give a positive loop, which confirms what we described above.  `` Figure 1 : Sketch of the feedback loop between the chemical, the wall stiffness and the diffusion coefficient. Figure 1 was generated using Claude but we thought it is nice to have a sketch of the feedback loop to support our answer to question 3. See the AI disclosure for information about how we used AI for this assignment.

Question 4 The pathogen divides when its area becomes larger than `rel_cell_div_threshold` times the base area, which is a fixed reference value of 1000 for all cells. The pathogen starts with an area of about 559, and in each simulation step its target area increases by 2. Since every step corresponds to 10 seconds (`rd_dt = 10`), a run of 2 hours is 720 steps, so the target area of the pathogen can only increase by about 1440, to a maximum of about 2000 (1999). We expected that a lower threshold would make the pathogen divide earlier and more often, and that more pathogen cells would make the infection spread faster. The default run: `rel_cell_div_threshold = 2`: (put images here from runs) Run 1: `rel_cell_div_threshold = 1`.  `` Run 2: `rel_cell_div_threshold = 3`.  `` In our runs, the infected region looked almost the same for all thresholds we tried (1, 3 and also 15), covering roughly the first two cell files after 2 hours. This can be explained by the numbers above: with a threshold of 3 or higher the pathogen would need an area of at least 3000 to divide, which it cannot reach within 2 hours, so it never divides and the threshold has no effect at all. Even with the default value of 2, the pathogen barely reaches the required area of 2000 by the end of the run. Only with a threshold of 1 (area larger than 1000) can the pathogen divide within 2 hours. Even then, the spread of the infection did not visibly change, which suggests that within this time the spread is mainly limited by how fast the chemical diffuses through the plant cell walls and not by the number of pathogen cells. To see a clear effect of the threshold, the simulation would have to run much longer, so that the pathogen reaches the division threshold several times.

Question 5 In the auxin model, which is the other model we worked with, all cells are treated the same way: no cell is ever prevented from rearranging its walls with its neighbours. The infection model is different, because it is the only model that uses the cell veto (*SetCellVeto* in *CellHouseKeeping*). With wall reconfiguration switched on (`compatibility_level = 65535`), wall elements can be moved from one cell to a neighbouring cell, so cells can gain or lose neighbours. Healthy cells set their veto to true, so their walls cannot be moved and they keep their neighbours. Cells that have been weakened by the pathogen's chemical set their veto to false, so only in the infected part of the tissue can walls shift and cells change neighbours. Whether a cell can change neighbours therefore depends on its state, which fits a pathogen pushing its way between weakened cells. In addition, the exchange of chemical between two neighbours depends on the wall they share: the diffusion coefficient is calculated from the stiffness of both cells' sides of that wall, while in the auxin model passive diffusion uses the same *D* for every pair of neighbours.

Question 6 The idea of the defense is that a plant cell that detects a lot of the chemical makes its walls stiffer instead of letting them be weakened. This would go in *CellHouseKeeping*, in the part that handles the plant cells, right after the infection level is calculated. The defense check needs to happen before the existing weakening rule, otherwise the weakening would overwrite it. We would also need two new parameters, a defense threshold and a defense stiffness that is higher than the normal value of 3. Pseudocode for *CellHouseKeeping*: for each cell *c*: if *c* is the pathogen: grow and divide as before, keep stiffness 3 else: level = chemical of *c* / 0.5, capped at 1.2 if level > defense_threshold: set stiffness of all walls of *c* to defense_stiffness keep veto on else if level > 0.1: set stiffness of all walls of *c* to 3 - level turn veto off else: set stiffness of all walls of *c* to 3 keep veto on Optionally, in *SetCellColor* we could give defended cells a different colour so we can see where the defense is active. This adds a negative feedback. With the defense, more chemical in a cell leads to stiffer walls. Since the diffusion coefficient is 0.00001 divided by the stiffness, stiffer walls lower the diffusion, so the chemical spreads more slowly to the next cells. So more chemical now leads to less spreading, which works against the positive feedback from question 3. The defended cells could act like a barrier that slows down or even stops the infection. Counting the interactions in the same way as in question 3: more chemical now increases the stiffness (a positive interaction), stiffness still has a negative effect on the diffusion coefficient (a negative interaction), and higher diffusion still increases the chemical in the neighbouring cells (a positive interaction). There is only one negative interaction, so the loop is negative. How did my teammate put the figures and images in the README file? I want to add some images for question 4 but I do not know how to do that.

Response: Your teammate used **GitHub's built-in file upload feature** directly inside the GitHub web editor (or a GitHub Issue/PR draft).

Whenever you drag and drop an image into GitHub's Markdown editor, GitHub automatically hosts the image on `[github.com/user-attachments/assets/]` (`https://github.com/user-attachments/assets/`)... and inserts the code into the text.

Here are the two easiest ways you can do the exact same thing:

Method 1: Directly inside the GitHub File Editor (Easiest)

1. Open your repository on **GitHub.com** and navigate to your `README.md`.
2. Click the **pencil icon** (Edit this file) in the top-right corner of the file preview.
3. Scroll to the spot under **Question 4** where you see (put images here from runs).
4. **Drag and drop** your image file from your computer directly into that spot in the text area (or copy your image and press **Ctrl+V** / **Cmd+V**).
5. GitHub will briefly show `[Uploading your-image.png...]` and will automatically replace it with an asset tag like:


```
! [image] (https://github.com/user-attachments/assets/...)
```

(Or an `` HTML tag if they resized it).
6. Scroll down, write a quick commit message, and click **Commit changes**.

Method 2: Resizing the Images (How your teammate got `<img width="..." ... />`)

Notice how your teammate's images look like this:

```

```

(Specifying only the *width* is usually best because the browser will automatically scale the height proportionally so the image doesn't look stretched).

Method 3: Committing Images into a Folder (Alternative Repo Approach)

If you are working locally in VS Code or Git instead of GitHub's web interface:

1. Save your screenshot inside your project repository (e.g., in a folder called `images/` or `assets/`, such as `images/q4_threshold_2.png`).

2. Reference it in your Markdown file using a relative path:

```
![Run default threshold 2](images/q4_threshold_2.png)
```

Or with custom width:

```

```

3. Commit and push both the image and the README.

User prompt: When using VirtualLeaf, where can I find the folders on my MacBook with the logs of every run? Because I cannot find them, I just have the VirtualLeaf program installed but I do not know where to find the folders and logs.

Response: By default, VirtualLeaf **does not automatically save folders, logs, or outputs** for every run. If you just run the simulation without changing the export settings, the data is not stored anywhere.

To get logs or outputs from your simulation runs, you have to explicitly tell VirtualLeaf to save them.

1. How to manually export data

If you want to save the state of a single run at a specific moment, use the top menu:

- **Export numerical data:** Go to **File > Export cell data**.
- **Save the model state:** Go to **File > Save Leaf** to save the current tissue geometry as an XML file.
- **Save images:** Go to **File > Snapshot** to save an image (PDF, PNG, etc.) of the current view.

2. How to automatically log data over time

If you want VirtualLeaf to continuously log data during the simulation (for example, taking a data sample every few steps), you must adjust the model parameters before running it:

1. In the parameters panel, look for **export_interval** and **export_fn_prefix**.
2. **export_interval**: Change this from 0 to how often you want to save data (e.g., setting it to 10 saves data every 10 steps).
3. **export_fn_prefix**: This determines the name of the files it will generate.

Where are the files saved on a Mac?

Mac applications are bundled as `.app` packages. If you just type a simple name like `my_run` in the `export_fn_prefix`, VirtualLeaf will likely bury the log files inside the hidden app bundle or the system's root working directory, making them very hard to find.

The best solution: Give VirtualLeaf an absolute path to a folder you can easily access. For example, create a folder on your Desktop called `VL_Logs`. Then, set your `export_fn_prefix` to: `/Users/noortje/Desktop/VL_Logs/run1_`

Now, when the simulation runs, your log files will reliably appear right in that folder on your Desktop!

(If you already exported files without a full path and are trying to recover them, you can try right-clicking the VirtualLeaf application icon in Finder, selecting **Show Package Contents**, and looking inside the `Contents/MacOS`, `bin`, or `data` folders to see if they were dumped there).

Sources

1. [Modeling Plant Tissue Development Using VirtualLeaf](#)

User prompt: How can I save it every 30 minutes in the simulation?

Response: To save data every 30 minutes of simulated time, you need to convert that time into simulation steps and set that number as your **export_interval**.

- **Calculate the steps:** Since 1 simulation step corresponds to 10 seconds (`rd_dt = 10`), 30 minutes (1800 seconds) equals **180 steps**.

- **Set the parameter:** In the parameters panel, change `export_interval` to 180.
- **Set the path:** Ensure `export_fn_prefix` is set to an absolute path like `/Users/noortje/Desktop/VL_Logs/run_` so the outputs do not get lost in the hidden app bundle.

You can verify this is working correctly by checking your destination folder as soon as the simulation passes the 180-step mark; the first log file should appear immediately.

User prompt: storage_stride is now 100 and xml_storage_stride is 500 but I get way too much output, even if I set export_interval to 180. (Because I want 1 export every 30 minutes.)

Response: The reason you are getting too much output is that `storage_stride` and `xml_storage_stride` are built-in VirtualLeaf logging parameters that run independently of `export_interval`.

Because your `storage_stride` is set to 100, VirtualLeaf is automatically saving data every 100 steps (about 16.6 minutes). This overrides or adds to whatever you are trying to do with the 180-step `export_interval`.

To get output strictly every 30 minutes (180 steps), you need to sync all of the saving parameters to the same 180-step cycle.

Change your parameters to the following:

- **export_interval:** 180
- **storage_stride:** 180 (This ensures numerical data is only logged every 30 minutes).
- **xml_storage_stride:** 180 (If you want the XML file saved every 30 minutes to take screenshots). *Note: If you do not need the XML files and only want the numerical data, set this to 0 to stop XMLs from generating entirely and save disk space.*

Let the simulation run past step 180. Are you now only seeing a single set of files generated at the 30-minute mark?

User prompt: rd_dt = 10, so how can I make sure that I get to save data every 30 minutes?

Response: Since `rd_dt = 10` means each simulation step represents 10 seconds of simulated time, you calculate the required steps like this:

- **30 minutes** × 60 = 1,800 seconds
- **1,800 seconds** / 10 seconds per step = **180 steps**

To ensure you only get output exactly every 30 minutes, you must set **all** of your saving parameters to 180 so they don't trigger at different times.

Change these settings in your parameters:

- **storage_stride = 180** (Saves the numerical log data)
- **xml_storage_stride = 180** (Saves the tissue geometry/visual state)
- **export_interval = 180** (If you are using a custom data export)

If you leave any of these at 100 or 500, VirtualLeaf will continue saving extra files at those intervals. Set them all to 180, and it will perfectly sync to a 30-minute schedule.

User prompt: I expected only documentation from 5 datapoints, but I got way more... So what is going wrong here?

Response: Are you seeing way more than 5 **files** in your output folder, or are you looking at a single log file and seeing thousands of **rows** of data?

If there are too many files:

- **The simulation ran past 2 hours:** 2 hours of simulated time is exactly 720 steps. If you did not manually click pause at step 720 or set a maximum step limit, VirtualLeaf will keep running indefinitely, generating a new file every 180 steps. Check the file names to see if the step numbers go past 720.
- **The GUI didn't save your input:** In VirtualLeaf, you must press **Enter** on your keyboard immediately after typing 180. If you click on another box without pressing Enter, the software silently reverts to its previous setting. Start a new run and verify the number in the box stays at 180 while the simulation is running.

If there are too many rows in one file:

- **Cell-by-cell logging:** VirtualLeaf logs data for *every single cell* in the tissue at each interval, not just one summary line per time step. If your plant tissue has 400 cells, a single 30-minute interval generates 400 rows of data. 5 datapoints will generate 2,000 rows. Check the first column (usually Time or Step) to see if the exact same time step repeats hundreds of times before changing to the next interval.

Which of these matches what you are seeing?

[Questionnaire: 6]

User prompt: Can I also get just 1 xml file for the entire simulation?

Response: In VirtualLeaf, a single XML file can only store a snapshot of the tissue at one specific moment in time. It cannot record the continuous history of the simulation in a single file.

If you only want one XML file showing the **final state** of the simulation at the 2-hour mark (step 720), you have two options:

- **Option 1 (Automatic):** Change `xml_storage_stride` to 720. Make sure to press **Enter** after typing. VirtualLeaf will only generate the XML file when it hits exactly 2 hours.
- **Option 2 (Manual):** Change `xml_storage_stride` to 0 (and press **Enter**) to turn off automatic XML saving entirely. Run the simulation, pause it at step 720, and go to **File > Save Leaf** to export the final state yourself.

If your goal is to collect numerical data (like cell sizes or chemical levels) for the entire 2 hours in a single document, you must rely on the numerical data log (`storage_stride`), rather than the XML files.

User prompt: Regarding the images in the README file, how can I make these images show up with a height of about 150-154 but scaled proportionally so that width and height are in the right proportions? `` `` `` `` ``

Response: You can do this either by **specifying only the height attribute** (letting the browser calculate the width automatically) or by setting the exact 16:9 scaled dimensions (`width="274" height="154"`).

Option 1: Set only height="154" (Simplest & Automatic)

If you remove the `width` attribute completely and only keep `height="154"`, the browser automatically scales the width proportionally so the image never stretches:

```
` `` `` `` ``

Response: `<img height="150" alt="D1E8FEEC-FACB-4251-9DD4-88EF194A0BB1" src="https://github.com/user-attachme
<img height="150" alt="6B2B1F5E-9E91-47BD-B76F-E0622611F77D" src="https://github.com/user-attachments/asse
<img height="150" alt="415906F7-79D0-48AB-B1F5-6DB5FA93ABFA" src="https://github.com/user-attachments/asse
<img height="150" alt="A0409D31-8586-4F77-8B02-3750A93E956C" src="https://github.com/user-attachments/asse
<img height="150" alt="B688D2F1-2245-4367-A698-8B988E5B6F2C" src="https://github.com/user-attachments/asse`